

Frame It as Cases

FL-02 — General AI Fluency, Week 2

VOICE CARD

Direct, honest about gaps, technical, no fluff.

Abdul Salam

Backend & AI Systems Engineer

The Case: FlyRank Backend & AI Engineering Internship

THE PROBLEM

The first task, BE-01, was never going to be hard on the code side. I'm already a Python developer, so two JSON endpoints in FastAPI wasn't a technical challenge. The real problem showed up somewhere the assignment didn't mention: getting the environment set up properly, and publishing the work the way the program actually expected it, one clean repo, one folder per assignment, real proof it runs.

That problem didn't go away after BE-01. It came back harder every single task after it.

WHAT I DID (AND DECIDED)

BE-01 hit a nested `.git` folder that made Git treat my own assignment directory as a broken submodule. Fixed it, then hit an HTTP 408 timeout on every push over HTTPS, three separate times, before switching the remote to SSH actually got the push through.

By BE-04, Play with Docker (the tool I'd planned to test everything on) got deprecated mid-assignment. I decided to move the entire testing workflow to GitHub Codespaces instead of forcing a dead tool to work, real Docker, in the browser, no local install.

That same task also needed a repository pattern: an in-memory store and a Postgres store both implementing the same interface, so swapping the backend meant changing one environment variable, not touching a single route. I proved persistence by killing the containers entirely and confirming the data survived, not just restarting the app.

By BE-06, the pattern repeated at a different layer: idempotency keys so a retried request wouldn't fire the same AI call twice, retry logic with real exponential backoff, and a loud alert log the moment a job failed for good instead of quietly dying in Redis.

Across every one of those, the decision I kept making was the same one: fix the actual failure in front of me instead of routing around it, and write down why, not just what.

WHAT CAME OF IT

One public repo, four real assignments deep, each one with working code, a README that matches the actual files (not a rewritten summary), and screenshots that prove it ran, not just that it compiles.

The repository pattern from BE-04 is the same discipline that makes BE-06's background job system swap-safe: the architecture doesn't just work, it's built so the next change doesn't require a rewrite.

Bio

Abdul Salam is a Backend and AI Systems Engineer, BSCS graduate from COMSATS University Islamabad, currently completing FlyRank AI's Backend and AI Engineering internship. His work centers on backend systems, Django, FastAPI, PostgreSQL, that integrate LLMs cleanly: RAG pipelines, background job processing, and architecture built so swapping a database or a model provider never means rewriting the service layer.

Contact / CTA

Look at the repo: github.com/salamlakhan7/FlyRank---Backend-Ai-Engineer---tasks

GitHub: github.com/salamlakhan7

LinkedIn: linkedin.com/in/abdul-salam-501b2025b

Email: salamlakhan7@gmail.com

Before / After: Cutting the Fluff

The exercise asks for one generic AI-sounding line versus the edited version in my own voice. Here's the actual edit, pulled from a real draft written for this internship's case study.

GENERIC AI LINE	EDITED VERSION
"Leveraged cutting-edge containerization technologies and implemented industry-best-practice architectural patterns to deliver a robust, scalable, and production-ready backend solution."	"I containerized Postgres with Docker, then proved the data survived a full container restart, not just an app restart, before calling it done."

The generic version could describe literally any Docker project ever written. The edited version can only describe this one, because it names the specific proof (a full container restart, not an app restart) that the generic line hides behind buzzwords instead of showing.